

Prague University of Economics and Business
Faculty of Informatics and Statistics



Edge Computing: Architecture and Applications

BACHELOR THESIS

Study program: Applied Informatics

Author: Jakub Šlambor

Supervisor: doc. Jiří Feuerlicht, Ph.D.

Prague, May 2023

Acknowledgements

I would like to thank doc. Jiří Feuerlicht, Ph.D., for his supervision of my thesis, as well as the co-founders of Herohero, who allowed me to conduct the case study at their company as well as helped me with the analysis.

Abstract

This thesis examines the topic of Edge Computing. The aim of this thesis is to investigate the application of Edge Computing to different application domains such as IoT, healthcare, etc., and to develop a prototype application that allows the comparison of Edge Computing solutions with the traditional approach. Edge Computing is first defined and categorized, the development and trends in the field are shown, and the hardware and software of a typical Edge computer are described, as is the security of Edge systems. A case study in a startup Herohero is conducted, and an application prototype using Cloudflare Workers is deployed to production, fulfilling the aim and answering the questions of what the purpose and business case characteristics of using Edge Computing are and what are the benefits and drawbacks of utilizing the Edge for the deployment of a Web application. The methods of processing used to achieve the goal of the thesis include the analysis of scientific documents and internet sources, comparison of the traditional and Edge approach, and gathering metrics to assess them both.

Keywords

edge computing, distributed systems, serverless, web application, CDN

Contents

Introduction	6
Methods of processing	7
1 Characteristics of the edge	8
1.1 Distributed and edge computing	8
1.1.1 Definition of edge computing	9
1.1.2 Goals of edge computing	10
1.2 The emergence of edge computing	10
1.3 Differences with traditional computing models	11
1.3.1 Client-server architecture	11
1.3.2 Cloud computing	11
1.3.3 Internet of Things	12
2 A brief history of digital transformation	13
2.1 Moore's Law and the Dennard Scaling	13
2.2 Digital transformation cycle	14
3 Components of an Edge system	15
3.1 Hardware	15
3.1.1 Hardware requirements	15
3.1.2 Examples of edge computers	15
3.2 Software	17
3.2.1 Edge Software specifics	17
3.2.2 Operating system	18
3.3 Edge Frameworks and services	18
3.3.1 EdgeX Foundry	18
3.3.2 Microsoft Azure IOT Edge	19
3.3.3 Vercel Edge Functions	19
3.3.4 Cloudflare Workers	21
3.3.5 Deno Deploy	21
3.4 Networking and communications	21
3.4.1 Near Range Communication	22
3.4.2 Long Range Communication	23
4 Edge applications use cases	24
4.1 Connected cities	24
4.1.1 Lidar-based smart city solutions	24
4.1.2 Autonomous Security Robots	24
4.1.3 Train Spotting	25
4.2 Healthcare	25

4.2.1	Dialysis AI model	25
4.2.2	Brain Stroke diagnosis	26
4.3	Agriculture	26
4.3.1	AI-Powered Tomato Pollinator	26
4.4	Web applications	27
4.4.1	Netflix Open Connect	27
4.4.2	Midterm Elections Results	28
4.4.3	AI Writing assistant	28
5	Security at the Edge	30
5.1	Types of Security Vulnerabilities	30
5.2	Examples of vulnerability exploits	30
5.2.1	Cloudflare parser bug	30
5.2.2	Mirai Botnet	32
5.3	Security Architecture	32
5.3.1	Hardware Security	32
5.3.2	Software Security	33
5.3.3	Network Security	34
6	Case study: Herohero edge prototype	35
6.1	Company background	35
6.2	Current technology stack	35
6.3	Analysis of possible approaches	35
6.4	Implementation	36
6.5	Cloudflare Workers results	37
6.5.1	Potential drawbacks	39
	Conclusion	40
	Bibliography	42
A	Cloudflare workers SSR code	47

Introduction

In 1991 at CERN, the first-ever web server was developed by Tim Berners-Lee. More than 30 years later, the World Wide Web is comprised of hundreds of millions of interconnected devices. Yet web developers are still often limited to designing the deployment of their applications on a single server in a large data center.

This is what Edge Computing tries to address. In the past, static assets were served by CDNs around the globe, close to users. Nowadays, a similar approach is being adopted for dynamic web applications, enabling widespread deployment to numerous locations. (Jiang, 2022) This, of course, is not the only area where Edge Computing can be applied. Common examples include the use in healthcare, the infrastructure of cities, manufacturing, and agriculture.

As defined in Making Sense of Edge Computing, at its most basic level, Edge Computing “*is the study of when to move data and when to move functionality.*” (Lea, 2023, ch. 1) A sub-area of distributed systems, Edge Computing is an approach in which computing and storage resources can be utilized more efficiently within a distributed infrastructure by deploying data processing components near the origin of data generation. This, in return, makes the system more scalable, secure, and cheaper. On the other hand, the complexity of developing and managing such a system is much higher, among other drawbacks. Both of these views are discussed further in the theoretical part of the thesis.

The aim of this thesis is to investigate the application of Edge Computing to different application domains such as IoT, healthcare, etc., and to develop a prototype application that allows the comparison of Edge Computing solutions with the traditional approach.

Edge Computing encompasses a diverse range of technical domains, spanning hardware design, communication systems, networking, Cloud Computing, Machine Learning, embedded systems, and security. The topic of Edge Computing, emphasizing its use in web applications, was chosen to comprehensively tie these domains together and to examine how an Edge system works.

The practical part of this thesis analyzes the current state of the code base of a technology startup, Herohero, where the author is the lead front-end developer. Based on that analysis, a suitable area of the application and its infrastructure was chosen, and a working prototype using edge computing techniques, as described in the theoretical part of the thesis, was developed. This prototype was then compared with the traditional approach based on multiple parameters, including developer experience, speed of the application, and security. The following questions were examined and answered in the process:

- What is the purpose of using Edge Computing, and what are the business case characteristics for which an Edge solution proves useful?
- In Web applications, what are the benefits and drawbacks of utilizing the Edge for deployment?

Methods of processing

In the theoretical part of the thesis, a scientific literature survey and analysis were used to gather information about the topic and to assess the current state of the technologies and discussions surrounding Edge Computing. Books, internet sources, and journal articles were all found using Internet search engines, including Google and Google Scholar.

The practical part of the thesis focused on implementing an application prototype in an internet startup, Herohero. There, a method of observation and subsequent analysis of the current state of the company's codebase was conducted, the prototype solution was developed, and then using the method of comparison, both versions of the app were judged in terms of performance and bandwidth usage.

1. Characteristics of the edge

When discussing edge computing, we must first define the edge, how it relates to other forms of information technology, and how it differs from the more traditional client-server and cloud computing models.

1.1 Distributed and edge computing

Edge computing is a fast-growing technology and research area that is finding its application in several industries. From a conceptual point of view, edge computing is a form of distributed computing.

Distributed computing dates back to the early 1970s with networks like the local area networks (LAN) and the first wide-area packet-switched network, ARPANET. (LivingInternet, 2020) To understand distributed systems, we must first briefly describe the fundamental theory behind networks. In his paper “*On Distributed Communications Networks*”, published in 1964, Paul Baran lays out distributed communication network concept in which each system component is connected to all adjacent components rather than to a few switching points, as in a centralized one. The third type of a system (or a network) in this categorization is a decentralized one, in which levels of hierarchy still exist to provide and accept control from other levels while keeping service-level autonomy. You can see a visual representation of a **centralized**, **decentralized**, and **distributed** network in Figure 1.1. (Baran, 1964)

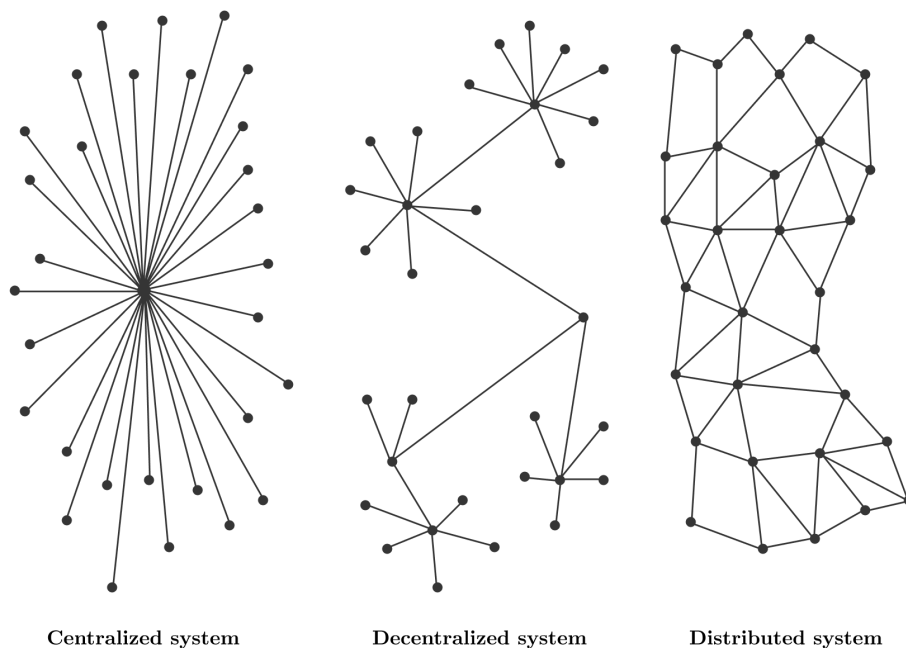


Figure 1.1: Types of network topologies (Source: Author’s visualization, based on Paul Baran’s categorization)

An example of a centralized system is a personal computer. The system is comprised of several different components, but without the central processing unit, or CPU, the system as a whole would be unusable. An example of a decentralized (hierarchical) system is a university or similar organization with a top-down approach to decision-making. At a university, high-level administrators and advisory boards assess key performance indicators and establish university-wide initiatives by liaising with the various colleges. The colleges, in turn, oversee and execute changes at the college level and communicate with departments. Finally, the departments implement any necessary modifications at the departmental level by collaborating with faculty and staff. Lastly, an example of a distributed system can be a cluster of computers that can calculate the first N Fibonacci numbers in parallel. As all edge systems are distributed, all examples of edge computing applications can be treated as examples of distributed systems. (Lea, 2023)

By the late 1990s, Akamai launched the first content delivery network (CDN) to resolve web congestion. This marks the start of the development of edge computing we know today. Despite its history, modern edge computing typically extends the cloud to devices that have never been connected to the Internet and is considered a novel paradigm.

1.1.1 Definition of edge computing

There exist many formal definitions of edge computing. Eric Hamilton of Cloudwards, one of the leading publications in the cloud technology world, wrote in 2018 that, at its simplest, *“edge computing is the processing and analyzing of data along a network edge, closest to the point of its collection so that data becomes actionable.”* (Hamilton, 2018) This Definition is echoed by Accenture, an IT services and consulting company, which defines it as *“a new capability that moves computing to the edge of the network, where it’s closest to users and devices — and most critically, as close as possible to data sources.”* (Tung, 2021) In other words, the edge aims to bridge the divide between cloud computing and data processing solely at the data source (or on-premise).

Some publications have a broader definition, stating that it includes any form of computing outside of a data center. However, this would include even the smallest embedded devices, such as home appliances or machinery controllers, which are not necessarily connected by networks and thus cannot be considered edge devices. Some argue that edge computing is a type of computing that mitigates latency, but reducing latency is just one of the issues addressed by edge devices. (Gill and Smith, 2018)

To summarize, as the book *Edge Computing Technology and Applications* puts it, *“edge computing is a form of computing services and hardware that provides processing, storage, and programmable control close to the sources of data generation or data consumption.”* (Lea, 2023, ch. 1)

1.1.2 Goals of edge computing

The basic goals of edge computing can be listed as follows:

1. To minimize the latency in processing, storing, or sending data.
2. To lower the cost of cloud and telecommunications by filtering data and consolidating data sources.
3. To provide security and data assurance. In some cases, data must be anonymized in order to be sent to the cloud safely and securely.
4. To connect simple sensors and non-IP capable networks to IP and internet capable networks.
5. To make computing and crucial communications more resilient in harsh environments, remote locations, and mobile devices.

These requirements cannot be fulfilled with today's infrastructure of cloud computing. (Amazon Web Services, Inc., 2023) In the 2010s, computers at the edge did not have any agency over their functioning. The central server decided when certain actions were to be performed, and the edge devices just awaited commands. With the emergence of small and affordable computers, this was about to change.

Today's edge devices can achieve the listed goals. They are capable of local processing of data, including sophisticated analysis and artificial intelligence. This minimizes latency and lowers the cost of cloud computing services, as the data is already aggregated, and a smaller load of valuable data can be sent over the network. (Bumgardner and Hickey, 2020, ch. 1)

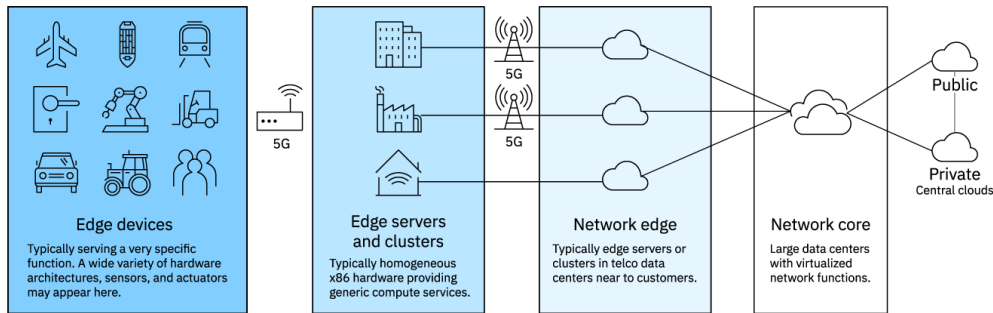


Figure 1.2: Contemporary topology of the edge (Source: IBM Developer)

1.2 The emergence of edge computing

If we look at the interest in the search term “Edge computing” on the Google Trends website, we can see a steady rise in interest beginning in 2016, and while reaching its peak in early 2022, the trend remains.

The emergence and continued interest in edge computing can be explained by looking at the problems it sets out to solve. Each year, about 20 % more data is generated than the

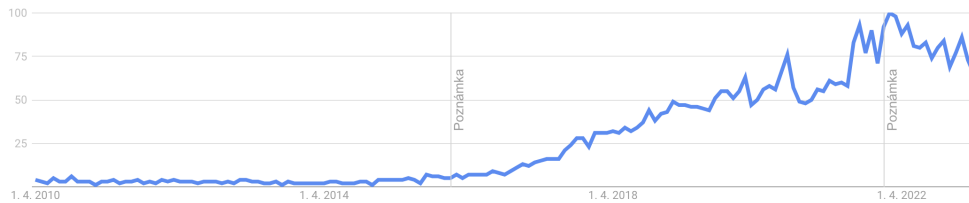


Figure 1.3: Interest in the search term “Edge computing”, 2010-2023 (Source: Google Trends)

previous year. In 2023 alone, it is estimated that 120 zettabytes will be generated worldwide. (Taylor, 2021) That equals roughly 120 billion terabytes. There are also more than 15 billion IoT-connected devices and sensors transmitting data. (Vailshery, 2022)

The requirements for fast and secure computing cannot be satisfied with existing cloud infrastructure. So, edge devices are found everywhere. They live in the harshest of environments and where communication and reliable infrastructure is something to be concerned about.

1.3 Differences with traditional computing models

1.3.1 Client-server architecture

The client-server architecture is a typical example of a centralized system. Users’ device connects to the central server, which processes the request and provides a response. In client-server systems, the majority, if not all, of the work is done by the central computer, server, or a group of servers. The users only interact as needed.

This approach is used in traditional web apps, email clients, and countless other applications. And for most of these, this is a perfectly suitable approach. But as the size of the data grows and the distance between individual nodes increases, some limitations start to arise.

1.3.2 Cloud computing

Edge computing is not meant to replace cloud computing. For the foreseeable future, the cloud and large-scale data centers will remain essential for the Internet as a whole. Native cloud applications often control and coordinate the “worker” edge nodes. The cloud is made for this purpose, as it can be managed centrally and easily scale with the application’s needs. (Bloomberg Media Studios and IBM, 2020)

Edge computers do not possess the same processing, storage, and networking capabilities as a machine found in a cloud data center. They are often smaller and remotely located. This is not a rule, however, as some edge devices can reside in a more traditional setting. An example of this is Netflix’s edge architecture which it uses to deliver its video content to a worldwide

audience without lengthy buffering. It comprises two main parts: cloud-based microservices for running the services that power the experience and a hardware-based infrastructure they call “Open Connect” to stream the bits as fast as possible. (Fisher-Ogden, 2021) This is further examined in Chapter 4.1.

Ownership is another distinguishing factor between cloud computing and edge computing. Companies and other private or public enterprises used to operate on-premise data centers that were owned and managed by said entities. This was technically inferior and less cost-effective than utilizing the services of a hyper-scale public cloud. So, as the cloud became readily available, companies migrated their services there. Edge systems do not share these characteristics. The reason is that edge systems are specialized and customized for specific purposes. Unlike cloud computing, no one-size-fits-all solution can be deployed across different industries or use cases. Some cloud providers are starting to offer edge computing solutions, such as dedicated hardware or software middleware. Still, most edge systems are tailor-made and designed to fulfill specific missions which cannot be packaged and sold as a cloud service would be. (Lea, 2023)

Finally, while a cloud system’s data has to reside in physical storage, based on the definition of cloud computing, it is typically not associated with one. On the other hand, edge systems are always connected to a particular location.

1.3.3 Internet of Things

Edge computing is not just another term for the Internet of Things (IoT). An **IoT device** is a physical object (such as a heart rate monitor, a fridge, or a thermostat) that is connected to the internet and can receive and transmit data automatically. These devices may not have a display or visible feedback and often rely on physical sensors and actuators. IoT devices are usually deployed on a massive scale and require low-power and small form factor designs, often consisting of low-cost intelligent sensors with limited computing capabilities.

While the majority of IoT devices can be categorized as edge computers because of their connectivity and intelligence, not all edge computing systems are IoT devices. An **edge device** is a computer directly connected to the internet with more computing power, storage, and networking facilities than a simple IoT device. It may or may not have physical sensors but is typically located near where data is consumed or gathered, like in the case of the previously mentioned Netflix edge data center, physically located close to the users to reduce latency.

Standalone **embedded systems** and micro-controllers also cannot be considered edge systems, as they are not connected to the Internet. Edge systems require communication in multiple directions. The so-called “north-south” network communication happens between the cloud and edge devices, while the “east-west” is the communication between the edge nodes. (Lea, 2023, ch. 1.4)

2. A brief history of digital transformation

Over the past fifty years, information technology has changed its architecture and structure in response to the limitations of computing technology and emerging use cases. With that, businesses have continuously digitized their products and services in a process called **digital transformation**. MIT Sloan Management Review states, “*Digital Transformation is better thought of as continual adaptation to a constantly changing environment.*” (Kane, 2017) The ultimate goal being changed customer expectations and new business opportunities. In other words, digital transformation is about how technology changes the conditions under which business is done, leading to changes in customer, partner, and employee expectations and the creation of new business opportunities.

2.1 Moore’s Law and the Dennard Scaling

Named after Gordon Moore, the co-founder of Intel, Moore’s Law is a famous observation that states that “*the number of transistors in an integrated circuit doubles every two years.*” (Mollick, 2006, p. 62) As chips became more powerful and cheaper, they found more use cases across the broader economy. In the beginning, chips were used primarily for defense systems, then for corporate computers, automobiles, and the Internet of Things.

When combined with the Dennard scaling, which states that “*as transistors get smaller, their power density stays constant,*” (Press, 2016) we can say that while the performance is improving with each generation, a problem with power consumption arises. This is especially noticeable in large data centers that use thousands of high-performance servers, consuming enough energy to power small cities. Even small battery-powered systems still face significant power consumption challenges. Figure 3.1 plots this relationship and illustrates the looming power constraints (notice the leveling off of single-threaded performance and frequency). (Rupp, 2018)

Moore’s Law and Dennard Scaling have driven the development of advanced microprocessors and silicon devices for high-performance servers and computers. However, the growth of edge computing and IoT has opened up new possibilities, even for devices that don’t require the most advanced silicon. This has made it economically feasible to connect remote devices via the Internet. Edge computing and IoT are proliferating, presenting new opportunities like the PC revolution in the 1990s and 2000s.

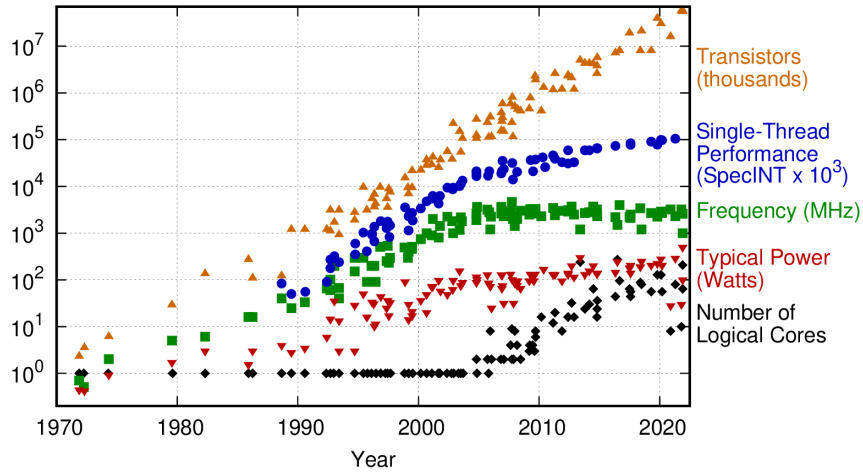


Figure 2.1: Microprocessor trend data, 1971-2021 (Source: Data collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten, and K. Rupp)

2.2 Digital transformation cycle

The first computers were called mainframes, and they were very costly, required significant square footage of specialized floor space, and could only perform a single task at a time. Mainframes were essential for transaction processing in banking, billing, and record management industries. They were usually connected to a set of “dumb” terminals, which resembled personal computers with monitors but were nothing more than a screen and keyboard. (Bashe et al., 1981) The IT industry has transitioned from hub-and-spoke systems centered around mainframes to distributed computing using PCs. With the cloud, the industry has returned to large, centralized computing working with smaller devices.

The IT industry undergoes shifts as new use cases and computing trends emerge. The trend again goes in the other direction with edge computing and IoT. Most microprocessors manufactured today are not used on personal computers, smartphones, or servers; they are exclusively used in embedded systems. And with Moore’s law, which allows hardware to scale exponentially in performance and decrease cost from generation to generation, we have witnessed the widespread adoption of electronics and embedded systems that bring intelligence, programmability, and connectivity to devices that previously did not possess such properties. (IBM, 2023)

While the first IoT devices (the first being an Internet-connected soda machine at the University of Carnegie Mellon) might have been nothing more than a novelty, they are a necessity in the modern age. And with the surge of their volume as well as the data they produce, the edge is what connects them, as it would be costly and ineffective to have them communicate with the cloud directly, and in some cases, downright impossible.

3. Components of an Edge system

Although the prototype application in the practical part of this thesis uses an edge networking framework, so no proprietary software or hardware needed to be developed, it is still beneficial to learn about edge hardware, software, networking, and protocols. This allows for the right selection of framework based on the application needs and gives an insight into the system's inner workings to understand better and measure its performance.

3.1 Hardware

3.1.1 Hardware requirements

The hardware that is used at the edge can vary greatly based on location, computational requirements, and environmental needs.

Going from the least computationally intensive, a smart home accessory, like a front door camera, will need only a low-powered processor that can handle simple facial recognition and alerting. Some processing might even be offloaded to the user's device or the cloud. Moving up the scale, a modern car has upwards of 100 computers controlling everything from power windows to engine diagnostics. Self-driving vehicles require even more computers, sensors, and power to process the driving data and the road telemetry in real time. Network connectivity is also crucial, especially in a moving vehicle. And as the vehicle can be driven to remote locations, endurance to harsh environmental conditions, like quickly changing temperatures, water, or dust, is also required. Still, it's processing data in an enclosed system.

An edge system aiding production in a manufacturing plant will need more computing power. Depending on the plant's product type, it might also need to be resilient to the elements. Typically, these systems will need to be operational at all times, requiring additional hardware as a means of backup.

A data center edge extension providing the data closer to users and businesses will need extreme computing and storage. But as they reside in an air-conditioned and protected building, they don't have to be concerned with elemental protection and ruggedness. The edge devices there will closely resemble those in a traditional cloud data center. (Lea, 2023, ch. 3.1)

3.1.2 Examples of edge computers

Raspberry PI is the most well-known single-board computer. It is typically used by hobbyists for IoT or edge computing-type projects. Its low cost, modularity, and open design make

it appealing to use at the edge for less intensive tasks. (Blues Wireless, 2021) Some smart home devices and appliances use a Raspberry PI or a similar system. If the task is more intensive and utilizes Machine Learning, for example, more powerful variations of single-board computers exist, like the **Orange PI** or **Odroid**. One real-life example is a portable “speed trap” that uses ML to identify vehicles, a radar sensor to measure speed, and then reports the data to the cloud.

NVIDIA Jetson and **Google Coral** are examples of more powerful chips accelerating the next generation of edge AI, robotics, and on-device intelligence. They can be used for prototyping, gathering real-world data at scale, and for production. Production use cases across industries include smart cities, healthcare, retail, agriculture, and more. (Mahler, 2023)

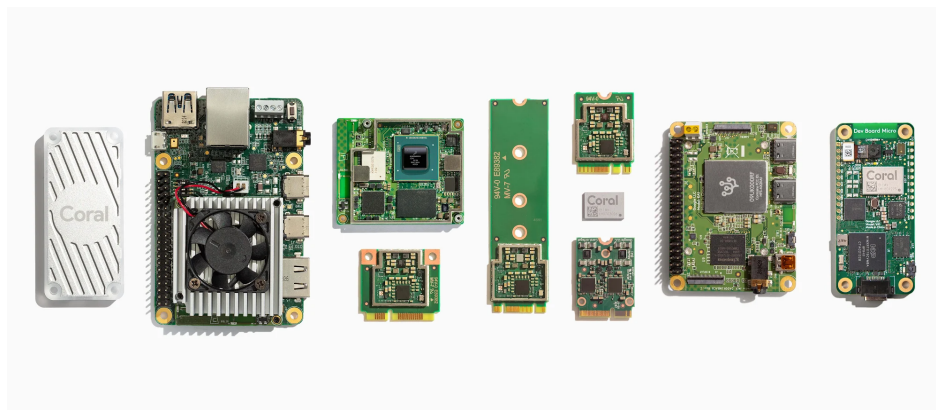


Figure 3.1: The Coral product lineup of edge hardware (Source: Google LLC)

Intel NUC is essentially a miniature computer. NUCs offer the flexibility of running various operating systems, such as Windows and Linux. Low power consumption is another advantage of edge computing, particularly in scenarios where energy efficiency is a priority. Intel offers a version of the NUC system called NUC Rugged for use in harsh conditions. This version is designed for applications that expose the hardware to dust and unconditioned spaces. They feature a fanless design with IP50 dust resistance and can operate at temperatures ranging from 0° to 40° C.

The fast food chain, Chick-fil-A, has employed the NUC system to run an Edge Compute cluster with Kubernetes at every restaurant location. The engineers there opted for an edge solution due to the physical space limitations (there is not a lot of space in the restaurant’s backrooms) and power outlet and networking constraints. The resulting system helps the kitchen with production processes and mobile payment terminals used in our Drive Thru. In total, they manage tens of thousands of edge devices across the US and Canada, delivering telemetry data from a wide assortment of smart appliances, like fryers and grills. (Chambers, 2023)

HPE Edgeline from Hewlett Packard is an enterprise-grade edge computing and processing system. It carries significant horsepower but also requires significantly more power. Systems such as the HPE Edgeline EL8000 Converged Edge System can be used in an edge data

center. A 2021 study of Florida-based startup OrbitsEdge suggests using such a system in the aerospace industry. With petabytes of data being generated in space daily and limited ways of getting them back to Earth, their goal is to “*bring high-power compute resources to low-earth orbit to enable data insights in space.*” (Hewlett Packard Enterprise, 2021)

Table 3.1: Edge systems CPU, GPU and Memory comparison (Source: Author’s work, manufacturers data)

System	CPU	GPU	Memory
Raspberry PI 4	4-core @ 1.8 GHz	—	1/2/4/8GB LPDDR4
Orange PI 5	8-core @ 2.4 GHz	600 MHz	4/8/16/32GB LPDDR4
ODROID-M1	4-core @ 2 GHz	650 Mhz	8GB LPDDR4
Coral SoM	4-core @ 2.3 GHz	integrated	1/2/4GB LPDDR4
Intel NUC 12 Pro Mini	12-core @ 3.3 GHz	integrated	16GB DDR4
Jetson Orin NX	8-core @ 2 GHz	918 MHz	8/16GB LPDDR5
HPE Edgeline EL8000	4-40 cores	optional	max. of 6TB DRAM

Table 3.2: Edge systems Power Supply, Connectivity and Price comparison (Source: Author’s work, manufacturers data)

System	Power Supply	Connectivity	Price
Raspberry PI 4	15 W	GbE, Wi-Fi, Bluetooth	\$35+
Orange PI 5	20 W	Gigabit Ethernet	\$75
ODROID-M1	24 W	Gigabit Ethernet, transceiver	\$90
Coral SoM	15 W	GbE, Wi-Fi, Bluetooth	\$99
Intel NUC 12 Pro Mini	35 W	Gigabit Ethernet, Wi-Fi	\$698
Jetson Orin NX	25 W	Gigabit Ethernet	\$699
HPE Edgeline EL8000	1500 W	10 Gb Ethernet	\$2000+

3.2 Software

3.2.1 Edge Software specifics

While the form and purpose of edge devices vary greatly, similar base software and functionality requirements exist. One of the most important aspects is security. Edge devices are often not protected from outside forces, either physical or virtual. Types of security vulnerabilities edge systems are susceptible to, examples of real-world attacks, and the ideal secure architecture is discussed in Chapter 6.

Edge systems can be found anywhere. Therefore the people servicing them have to be able to access, manage, and control them remotely. After the initial setup, further updates and adjustments of such a system should also happen automatically. Moreover, it is important to

take into account the health of a computer that is located remotely. This entails the proper setup of logging, account management, and remote updates.

Since edge computing systems can be responsible for managing critical infrastructure, they must be highly reliable and robust. To achieve this, failsafe and fallback solutions are essential. Redundant components are often employed in edge computing systems to prevent critical failures. For instance, the software should be able to detect the loss of the primary network connection that provides internet communication and either start saving the data locally and send it out once the connection is restored or redirect data via an alternate connection, such as a mobile network. (Lea, 2023, ch. 4)

3.2.2 Operating system

When it comes to choosing an operating system for edge systems, most devices will opt for a modern operating system such as Linux (for example, Yocto Linux, OpenWRT, Ubuntu/Ubuntu Core, RaspberryPi (Debian), or RedHat Linux) or Google Android. Others may choose a commercial OS like Windows IOT from Microsoft. The decision to use one operating system over the other depends on the specific use case. For instance, if data is being transferred to Microsoft Azure, using Windows IOT with extensions to manage the edge device and transfer data to and from Azure servers might be a logical choice. On the other hand, a Linux derivative may offer more flexibility in design choices and be better suited to accommodating nuanced hardware requirements.

If the edge system is intended to function as a remote server for content delivery, it is unlikely to be compatible with a lightweight real-time operating system (RTOS) or an operating system specifically designed for Internet of Things (IoT) devices. Instead, it will typically require a fully virtualized environment and hypervisor similar to those found in large-scale data centers. (Lea, 2023, ch. 4.1.6)

3.3 Edge Frameworks and services

Not unlike how operating systems serve as a base for other software, frameworks offer a wide range of functionalities and services to facilitate the setup of edge systems. Organizations like the Linux Foundation, Microsoft, and Amazon, offer frameworks that include essential components such as the operating system, remote management, security, and cloud communication.

3.3.1 EdgeX Foundry

EdgeX Foundry is *“a highly flexible and scalable open-source edge platform that facilitates interoperability between devices and applications at the IoT edge.”* (The Linux Foundation,

2023) The primary goal of EdgeX is to offer a hardware and vendor-independent framework that facilitates the quick development of software for edge and IoT devices. EdgeX was initially launched in October 2017 and has received support from numerous organizations, including IBM, ARM, ATT, Dell, Qualcomm, Red Hat, HP, and Samsung.

The main features of EdgeX are an inbuilt firewall and encryption, a litany of network and communication protocols, and services to enable the operation of an edge device even when network connectivity is disrupted. This means that even if the device loses all network connections, it will continue to gather data and perform its functions. Once connectivity is restored, the device will automatically re-establish communication with other systems. Edge X also provides the ability for engineers to selectively choose which data is processed locally and which data is propagated through the Internet to the cloud. This filtering feature can help in reducing transmission and cloud costs. (The Linux Foundation, 2023)

3.3.2 Microsoft Azure IOT Edge

While EdgeX Foundry is an open-source solution that provides a robust starting framework for enabling edge devices, it's worth contrasting it with commercial frameworks such as Microsoft's Azure IoT Edge. **Microsoft Azure IOT Edge** offers many similar features to EdgeX Foundry, but with some notable differences. The Azure IoT Edge framework provides a scalable solution for managing, securing, and ensuring reliability for millions of edge devices. It enables module containers to run on both edge nodes and within Azure for intensive computational processing using cloud connectivity. Similar to EdgeX Foundry, all software is deployed as containers and delivered to edge nodes in a controlled manner. (Microsoft, 2023)

3.3.3 Vercel Edge Functions

Frameworks like EdgeX Foundry and Azure IOT Edge are not concerned with any particular use case but rather standardize the Edge connectivity and communication with the cloud. As this thesis is focused on the use of the Edge in modern web applications, we will now explore the most popular frameworks and services in this space.

Vercel, a San Francisco-based company, is the leader in frontend development and deployment and is also the developer of the most popular full-stack React framework, Next.js. Next is an open-source framework used by the world's largest companies, including Airbnb, Netflix, TikTok, and many more. One of the things that make Next.js so popular is Vercel's vast Edge infrastructure, which makes it possible to quickly deploy content and run compute functions close to the users. Vercel's Edge Network is *"both a Content Delivery Network (CDN) and a globally distributed platform for running compute at the edge."* (Vercel, 2023)

When the request originates from the browser, it first goes through **Edge Middleware**,

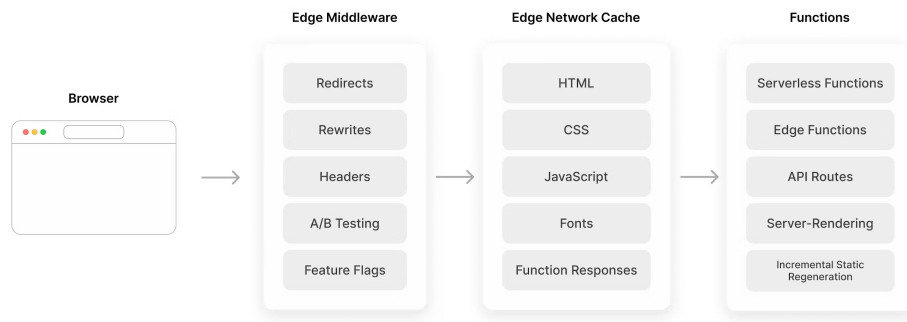


Figure 3.2: Vercel’s Edge Network infrastructure (Source: Vercel)

which is essentially the code that executes *before* the request gets processed. Based on the request, the response can be modified. Redirects can be used to direct the client to a different URL or to show the secure version of the site. Rewrites can change the requested URL behind the scenes, so the server can process it dynamically, making personalization possible.

Serverless Functions can be leveraged by developers to create functions using languages such as JavaScript, among others, to handle tasks like user authentication, form submissions, and database queries. “Serverless” in the name is technically a misnomer, as a server is still involved. The name points to the fact that a dedicated server is not needed to run the function and, instead, is only activated when a request is made. Some limitations exist, such as the maximum size, execution time, and memory a given function can use.

Edge Functions handle the same tasks as their Serverless counterpart, with some key differences. Edge Functions run on the more performant and cost-effective Edge Runtime, which is built on Chromes V8 engine. Edge Functions are also deployed globally and execute in the region nearest the user. On the other hand, the distance between the data source and where the function runs may add undesirable latency to the request. The maximum size, execution time, and memory limits are also stricter with Edge functions.

Table 3.3: Functions comparison (Source: Vercel)

Factor	Serverless Functions	Edge Functions
Cold starts	Yes	No
Regions	Single	Deployed globally
Runtime	Node.js, Go, Ruby, Python	Edge Runtime
Node.js APIs	Yes	Limited
Max size	50MB	1-4MB
Max execution time	10-900s	30s
Max memory	1024MB	128MB

3.3.4 Cloudflare Workers

Cloudflare Workers is a serverless execution environment that operates on Cloudflare’s world-wide network, spanning over 200 cities. It is similar to Edge Functions in that they also deploy globally and can run compute close to the end-user. In contrast, they can run more programming languages than Vercel, for example, Rust, C, and C++. They also offer cold starts with no delays, a command-line interface called Wrangler, and can interact with other Cloudflare services, like their key-value database. (Cloudflare, 2023b)

3.3.5 Deno Deploy

Deno is a popular alternative to Node.js. It is a JavaScript, TypeScript, and WebAssembly runtime that is, like Vercel’s Edge Runtime, based on the V8 engine. Deno was co-created in 2018 by Ryan Dahl, who previously developed Node.js. It tries to address the inadequacies of Node.js, like requiring a separate package manager.

Deno Deploy, a serverless deployment platform, was inspired by Cloudflare Workers (Dahl, 2022) and is similar to Vercel’s Edge Network. It was initially released on June 23, 2021, and features 35 regions worldwide. Similar to Vercel’s Next.js, Deno has also released a frontend framework in 2022 named Fresh. (Deno, 2023)

Table 3.4: Cloudflare Workers and Deno Deploy comparison (Source: Cloudflare, Deno)

Factor	Cloudflare Workers	Deno Deploy
Cold starts	Yes (Instant)	No
Regions	200+ globally	35 globally
Runtime	JavaScript, Rust, C, C++	JavaScript
Node.js APIs	Limited	Yes
Max size	5 MB	20 MB
Max execution time	30s	50ms
Max memory	128 MB	512 MB

3.4 Networking and communications

No Edge system is complete without communication with the outside world. The main forms of networking Edge devices typically perform is communication with the Cloud and in between Edge devices themselves. Selecting the appropriate communication medium and protocols is essential for developing a secure, resilient, and economically feasible edge solution. The speed and bandwidth, range, reliability, and power consumption are all differentiating factors in communication standards.

Networking can either be wired or wireless. Wired solutions like Ethernet, fiber-optics, or industrial protocols like ModBus are fast and resilient forms of networking. However, they require a physical connection, a wire, which for many applications, especially when talking about IoT and the edge, is not economically or geographically feasible. Edge devices most commonly use wireless forms of communication, which can be classified by their typical speed and maximum range. (Lea, 2023, ch. 5)

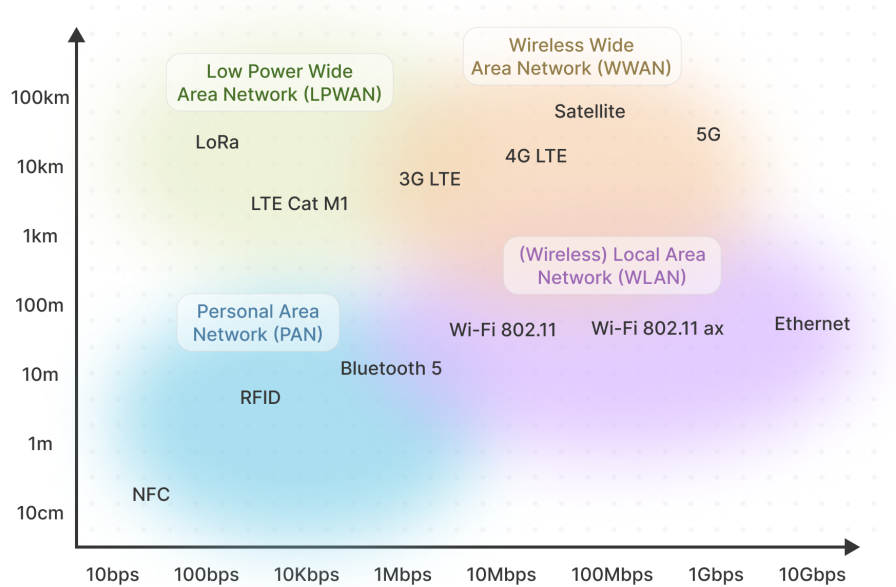


Figure 3.3: Communication standards arranged by maximum speed range (Source: Vercel)

3.4.1 Near Range Communication

Near Range Communication is communication that happens in a range of up to 250 meters. The two main categories in this class are the Personal Area Network (PAN) and Wireless Local Area Network (WLAN).

Personal Area Networks are classified as extremely short-range systems. Usually, they can have a range spanning from centimeters to a maximum of about 100 meters in rare instances. PAN systems encompass well-known protocols such as Bluetooth, NFC, or RFID and find widespread use in wearable IoT applications like heart rate monitors. PAN protocols are designed to consume minimal power, enabling battery-based sensors, or they can even operate without their own power source and instead draw energy from radio waves. It's important to note that most PAN communication systems do not support the TCP/IP protocol.

Wireless Local Area Networks include well-known protocols, like Wi-Fi (802.11). Both Wi-Fi and its wired counterpart, Ethernet, are TCP/IP compatible. WLAN systems offer bandwidths ranging from 100 Mbps to over 1Gbps, with newer protocols like 802.11ax improving performance. However, the range is restricted to approximately 100 meters. The first version of Ethernet featured a bandwidth of 3 Mbit/s. Today, a standard ethernet connection can

achieve speeds of up to 400 Gbit/s, with new versions with rates up to 1.6 Tbit/s under development.

Typical uses of Near Range Communication in Edge devices include situations where laying fiber or cable would not be feasible. Imagine a parking garage where a system indicating the number of free spots is being installed. Each parking spot has to have a sensor to detect if it's occupied. The building might have been built decades ago, so connecting the whole system together requires the use of networking, like Wi-Fi or Bluetooth. And systems like smart home devices, such as a smart lightbulb, often use a proprietary protocol called Zigbee, which is similar to Bluetooth but aims to be more affordable and cost-effective. (Lea, 2023, ch. 5.4)

3.4.2 Long Range Communication

When long-range communication is needed, meaning distances over 250 meters, either Wireless Wide Area Network (WWAN) or Low Power Wide Area Network (LPWAN) can be used, based on the bandwidth requirements.

LPWAN technology offers two crucial features for edge and IoT systems: extensive coverage range and low power consumption. However, there is a tradeoff in terms of bandwidth and transmission speed. Many LPWAN protocols are proprietary and necessitate licensing. The most commonly used LPWAN variants are LoRa and LTE Cat M1.

Cellular or satellite communication is used when higher bandwidth is needed. 5G is the current generation of cellular communications. Unlike earlier cellular models, 5G places less emphasis on hardware and radio components, instead prioritizing software control, routing, and software-based security. Previous cellular systems, such as 4G LTE, relied on specialized hardware provided by companies like Ericsson and Huawei to manage all communication between cell towers and central offices. In contrast, 5G adopts a more generic approach, leveraging software-defined services that offer increased flexibility and cost-effectiveness.

Cellular-based edge systems are extensively employed by various emergency response vehicles. These systems serve to provide internet connectivity and global positioning services. Similarly, cellular edge computing is utilized in waste management systems. Smart garbage bins communicate with collection trucks, notifying them when bins are full or require servicing. The lack of reliance on wired internet connections or power sources is making it highly convenient. (Lea, 2023, ch. 5.6)

4. Edge applications use cases

This chapter presents examples of edge computing applications across a wide range of industries.

4.1 Connected cities

4.1.1 Lidar-based smart city solutions

Bluecity is a Montreal-based artificial intelligence software company (which, at the time of writing, has been acquired by Velodyne Lidar) specializing in Lidar-based smart city solutions. Lidar uses a laser to determine an object's distance from the measuring device. This is achieved by measuring the elapsed time between Lidar sending out the laser signal and receiving it back after it has been reflected by the targeted surface. Common uses include construction, archeology, and geology. Recently, autonomous vehicles have been using Lidar in combination with traditional camera-based sensors to paint an accurate picture of the vehicle's surroundings, street signs, and other vehicles and pedestrians on the road. Studies have indicated that accidents are much more likely to occur in bad weather conditions. Lidar works independently of atmospheric conditions. Its sensing is not visual, so it can be effectively used in understanding and evaluating traffic data, providing valuable insights to city planning departments. This includes identifying problematic intersections, minimizing congestion, promoting smart planning, and reducing emissions.

Bluecity's systems utilize the NVIDIA Jetson edge AI platform, as it offers a compact and energy-efficient module with GPU-accelerated computing capabilities. Running a proprietary 3D perception software, the edge computing system enables the traffic management solution to detect all road users by processing up to 50 lidar frames per second in real time. These solutions are already being used in several Canadian, American, and European cities, including Irvine, Austin, Texas, Boca Raton, Trois-Rivières, and Helsinki. (Sinha, 2022)

4.1.2 Autonomous Security Robots

DOGU, a South Korean company, focuses on AI, autonomous technology, and complete robotics platforms. DOGU's product range includes Autonomous Security Robots (ASRs), providing extensive indoor and outdoor coverage. As of November 2022, DOGU operated 29 ASRs in the country and was looking to expand further over the following two years. Indoor robots typically operate in factories, hospitals, shopping malls, or parking lots. The outdoor variant is used in factories and in parks. The robots integrate with computer vision AI, thermal AI, sound AI, gas detection, and video streaming. The robots use reComputer

J2021 edge system, 3D Lidar of mapping and navigation, and reSpeaker USB microphone array for voice pickup. (NVIDIA, 2023)

4.1.3 Train Spotting

Rail Vision is an Israeli startup that makes obstacle detection and classification systems for the global railway industry. “The systems use advanced electro-optic sensors to alert train drivers and railway control centers when a train approaches potential obstacles — like humans, vehicles, animals or other objects — in real-time, and in all weather and lighting conditions,” says Shahar Hania, the co-founder of Rail Vision.

Trains require a lot of distance to stop once the brakes are applied. If a driver sees an obstacle on the tracks, it may be too late to avoid it. For instance, a train traveling at 160 kilometers per hour takes about 800 meters to stop. However, Rail Vision systems can detect obstructions on tracks up to 2 kilometers away, giving the conductor alerts with visual and acoustic signals. This gives more than 20 seconds to react and decide whether to brake. Additionally, the edge system can be integrated directly with the train, breaking without human interaction if the system determines that the threat is severe enough. (Lee, 2022b)

4.2 Healthcare

4.2.1 Dialysis AI model

According to the U.S. Department of Health and Human Services, Taiwan has the world’s largest incidence of treated End Stage Renal Disease (525 per million population in 2020), requiring permanent kidney replacement therapy. (USRDS, 2023) Taipei Veterans General Hospital (TVGH) is working to improve outcomes for patients with kidney dialysis. They utilize an AI model that can predict the risk of heart failure in real time during the procedures. Clinicians are provided with crucial risk prediction factors on a dashboard. Risk factors include the wrong measurement of dry weight or the patient’s weight without any excess fluids in the body.

TVGH uses a risk assessment tool that analyzes up to 200 sets of dynamic physiological signs and medical records, blood test results, and medication information. The tool employs multiple AI algorithms, such as decision trees, gradient boosting, and convolutional neural networks, to detect abnormal patterns in the patient’s data. The tool was trained on a dataset of 3 million health records. Recently, they also added more indicators to the tool, such as hemoglobin levels and chest X-ray image analysis, to improve its predictive capabilities.

Citing Professor Der-Cherng Tarng, chief of the department of medicine at TVGH, “*By deploying NVIDIA Jetson next to each dialyzer to perform AI prediction during the procedure, we can achieve real-time insights in a way that’s affordable and effective, even for small-scale*

dialysis centers.” With the help of edge computing, the AI risk assessment model achieves 90 % accuracy. And there are plans to expand the testing of the software to other Taiwanese hospitals. (Flores, 2022)

4.2.2 Brain Stroke diagnosis

For every minute that a stroke goes undetected, the average patient loses nearly 2 million neurons. (Saver, 2006) Meaning that for each hour in which treatment is not provided, the brain loses as many neurons as it does in more than three and a half years of normal aging. EMVision, an Australian healthcare technology developer, is creating a new, portable device capable of pre-hospital stroke diagnosis. The electromagnetic imaging technology it uses can be utilized to distinguish between ischemic strokes, in which the blood supply to an area of the brain is lost, and hemorrhagic ones, which are caused by internal bleeding.

“Whether for a new, acute stroke or a complication of an existing stroke, urgent brain imaging is required before correct triage, treatment or intervention decisions can be made,” says Ron Weinberger, CEO of EMVision. The powerful edge computer inside the helmet can handle complex AI modeling and send critical data back to the hospital for further analysis. (Lee, 2022a)

4.3 Agriculture

4.3.1 AI-Powered Tomato Pollinator

There are about half a million acres of greenhouse tomato crops in the world. Not only do the plants require soil, water, and sunlight, but they also require assistance in transferring pollen to the female organ of the flower to initiate the process of pollination. Usually, this task is performed by bumblebees flapping their wings, causing the pollen to become loose.

Israel-based startup Arugga is building a robot that takes care of pollination. The robots equipped with AI use computer vision to identify the flowers that are ready for pollination. They then generate air pulses to mimic bumblebees and stimulate the pollination process. This also does not interfere with the natural cycle, as some bumblebees are still present in the greenhouse, and the ones that are no longer needed would be artificially manufactured by the growers.

Arugga’s robots are constructed using readily available carts and incorporate NVIDIA Jetson Xavier NX modules. These modules empower the robots to run algorithms at the edge, facilitating the execution of AI capabilities directly within the robots themselves. In the future, the company plans to integrate more AI modules into the robots, enabling them to undertake additional tasks. They envision the robots being capable of detecting pests and

diseases, providing treatment to distressed plants, and applying solutions to enhance overall plant health. (Kontzer, 2021)



Figure 4.1: Arugga's AI robot, computer vision in the top left corner (Source: Arugga AI Farming)

4.4 Web applications

4.4.1 Netflix Open Connect

Back in 2007, when Netflix first introduced online streaming, the system behind it was quite simple. A hardware load balancer, a monolithic web app, and a relational database all running in one data center were enough for thousands of users watching back then. Nowadays, Netflix is used by over 230 million members, and roughly 15 % of downstream global Internet traffic is generated by the video streaming service Netflix, the highest figure among all websites (Armstrong, 2023). The sheer amount of data Netflix users stream left engineers with a difficult task. Users want their videos on demand, synced seamlessly across devices, with little to no buffering.

Netflix's overall architecture is a dichotomy. Cloud-based microservices power the experience and provide adaptability and scalability to the components of the ecosystem that require them. Hardware-based Open Connect infrastructure streams the data efficiently, making it an essential component of Netflix's global scale. The Edge in Netflix's cloud architecture serves as the main entry point to the microservices ecosystem. It is the starting and ending point for every request. Highlighting the critical need for resilience; if any aspect of the edge is offline, so is Netflix. The edge architecture comprises four fundamental layers.

The first layer is the Cloud Gateway, also known as Zuul. Zuul is a gateway service that provides:

- Authentication to determine which user is sending the request.
- Routing to analyze the URL, incoming parameters, and configuration to direct the request to the right origin service.
- Insights that measure request timings, report success/failure metrics, and record tracing information for debugging purposes.
- Rate Limiting certain requests to prevent overloading of the system.

The next layer is the BFF (Backend for Frontend) layer, which houses device-specific logic and data transformations. Beyond the BFFs, there is a comprehensive set of APIs that encompass the entire customer journey on Netflix. These APIs handle actions such as signing up, logging in, managing profiles, browsing or searching for content, playing media, and more. (Fisher-Ogden, 2021)

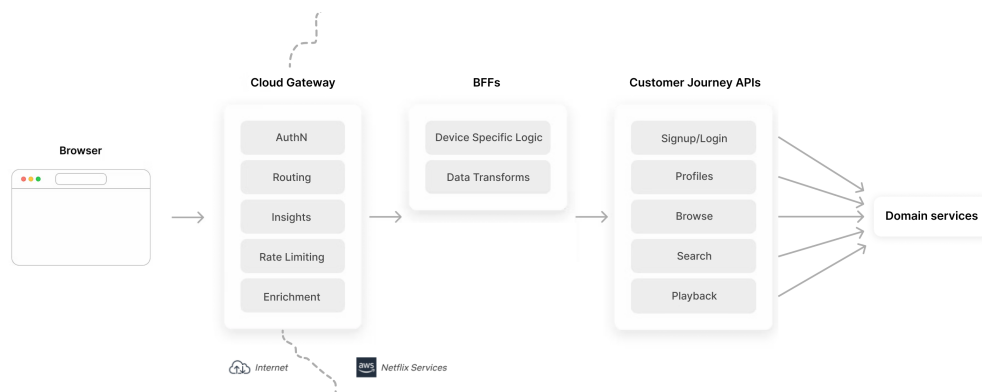


Figure 4.2: Netflix's Cloud Edge Architecture (Source: Netflix)

4.4.2 Midterm Elections Results

The Washington Post utilized Next.js to deliver the US Midterm Elections Results. Their engineering team recognized the advantages of the serverless model in production and found that when combined with Vercel's Edge Network, it offered exceptional performance and scalability to handle high-traffic periods effectively. The Washington Post used the Edge model to handle the high-traffic moment flawlessly, making it “the smoothest election night anyone could remember,” according to the Director of Newsroom Engineering, Jeremy Bowers. (Rauch, 2022)

4.4.3 AI Writing assistant

Molin is an AI-powered writing assistant that enables users to write with remarkable speed, offering a collection of useful templates for various writing needs, such as blog posts, emails, product descriptions, and more. One of Molin's standout features is its support for foreign languages, catering to over 3,000 users from non-English speaking countries.

Cloudflare’s platform played a crucial role in enabling them to swiftly bring their product to the market. It provided them with the ability to seamlessly scale from zero to thousands of concurrent users. Molin operates entirely at the edge of Cloudflare’s scalable Workers platform, utilizing caching to accelerate page loads, HTMLRewriter to make pages dynamic and enable real-time translation, and KV as the sole database. (Cloudflare, 2022)

5. Security at the Edge

Security at the Edge should not be an afterthought. On the contrary, it should be the foundation of every successful Edge system. The surface area of attack means the exposed elements of a system that an attacker can target. Edge devices are unique as their surface area is significantly larger than in other systems.

5.1 Types of Security Vulnerabilities

The Edge is the most attractive medium for recent systemic attacks on infrastructure. The number of Edge devices has grown exponentially, as opposed to the number of servers, personal computers, and other devices, whose growth has largely stayed the same or even declined. Edge computing is also a relatively young field, so gaps in security are likely. Therefore, Edge systems are considered a weak point.

The three main modes of attack on Edge devices are:

1. **Penetration attacks.** This includes any attack by a third party that tries to infiltrate the system by using a backdoor or stolen credentials.
2. **Ransomware attacks.** These attacks plan to encrypt or hijack personal details, records, data, and photos of a user. The user is forced to pay the attacker (using cryptocurrency for anonymity), threatened by the potential of losing valuable data.
3. **Distributed Denial-of-Service Attacks (DDoS).** The goal of this type of attack is to flood a system with so much traffic that it becomes overwhelming, causing a target's infrastructure to slow down or prevent users from accessing a service altogether. The attack usually involves flooding a target system with seemingly harmless requests but doing so with millions of agents accessing the system at the same time.

It is important to understand that no system can be 100 % secure. Even an enclosed box with no network connection can still be powered down, for example. The goal of the system architect should be to minimize the attack surface as much as possible.

5.2 Examples of vulnerability exploits

5.2.1 Cloudflare parser bug

Cloudflare is an American company that offers services such as a content delivery network (CDNs), cloud cybersecurity, DDoS protection and is also an ICANN-accredited domain registrar.

On February 18, 2017, work began winding down this Friday afternoon at the Cloudflare HQ. Then, at 4:11 pm Pacific time, Tavis Ormandy from Google’s Project Zero, a team of security analysts, discovered a severe issue with Cloudflare’s system which suggested a possible widespread data leak. The issue concerned Cloudflare’s CDN service, which has many points of presence across the world with edge servers that cache content from the origin server. When a user makes a request for a particular website, the request is directed to the nearest edge server, where the content is most likely already cached. It was here that Cloudflare not only returned the requested website but also cookies, keys, and other sensitive customer data. To make matters worse, search engines like Google also regularly index and cache websites, so this leaked data could also be accessed through Google’s cache.

The engineers noticed in the dashboards that the occurrence of this bug seemed to correlate with the usage of the email obfuscation feature, which was using a new HTML parser. By 5:22 PST, about an hour after the initial report, email obfuscation had been disabled worldwide. However, the bug was still occurring, as another two features were found to be problematic: automatic HTTP rewrites, and server-side excludes.

Many of Cloudflare’s services rely on parsing and modifying HTML pages as they pass through their edge servers. This was the clear common denominator among the three aforementioned services. Furthermore, these three features all used the new HTML parser mentioned earlier: cf-HTML. The engineers, however, found nothing suspicious in the code despite thorough verification.

Cloudflare had originally been using a parser called Ragel. The HTML web page consumed by the Ragel parser is represented by a series of data buffers, with each buffer containing a portion of the HTML code. But in the generated code, there was an issue that would allow the pointer to overrun the end of the file and leak confidential information which lay behind the end of the HTML file itself. If the following check has contained `>=` instead of `==`, jumping over the buffer end would have been caught.

```
/* generated code */
if ( ++p == pe )
    goto _test_eof;
```

The Root cause of the bug has been found to be a faulty implementation of the Rachel parser, which before the migration, remained hidden. Two days after the incident, pointer checks to detect memory leaks were rolled out, and three days later, the engineers determined it was safe enough to re-enable the three aforementioned features. Cloudflare then worked with the various search engines to purge their caches of affected websites. (Graham-Cumming, 2017)

In terms of overall impact, evidence suggests that it was quite small. There were quite a few conditions that needed to be met for the bug to manifest, and Cloudflare claims that there is no evidence of the bug being leveraged for any attacks. We know that only 0.06 percent of Cloudflare websites were affected but that the bug occurred more than 18 million times. (Prince, 2017)

5.2.2 Mirai Botnet

Perhaps the largest exploit involving Edge devices is called Mirai. Mirai is malware that infects smart devices that run on ARC processors, turning them into a network of remotely controlled bots. This network called a botnet, can then be used to launch DDoS attacks.

In September 2016, the creators of Mirai first tested the malware by performing a DDoS attack on the website of a prominent security expert. A week later, they made the source code public, perhaps to conceal the true origin. This code was soon copied by other cyber criminals and is believed to have caused the extensive attack that drove the domain registration services provider, Dyn, to go offline in October 2016, bringing huge sites like Netflix and Amazon down with it.

The special trait that made Mirai unlike any other attack prior was that it worked as a self-replicating and self-propagating worm that actively searched for and found vulnerable IoT and edge devices to conduct the attack. After infecting a device, the worm would scan the internet accessible by that device to locate other potential hosts. Infiltrating a device was not sophisticated. Mirai employed a database of only sixty-four commonly used logins and passwords to try to gain entry into a system.

On its initial day, the malware located and infected 65,000 systems. In just three months, it managed to identify and exploit 600,000 attack vectors. The devices infected ranged from Linux-based IoT devices, such as Wi-Fi routers, web cameras, air quality monitors, and set-top boxes. Denial-of-service attacks are quantified based on the volume of data being generated to inundate a target's infrastructure. The Mirai attack is currently the largest and most potent attack ever recorded, with 623 gigabits of malicious data being generated every second, while only about 100,000 of the 600,000 infected devices were used to perform the attack. (Cloudflare, 2019)

5.3 Security Architecture

5.3.1 Hardware Security

The security of an Edge device starts with the physical security of the device itself. As Edge devices do not have the luxury of residing inside a well-secured and monitored data center, there is always the possibility of a physical attack, be it unauthorized access of the device without a proper card or a unique key, through damaging the enclosure to gain entry, or stealing the device entirely. The device should be able to detect such occurrences, and if its chassis has been opened or tampered with by an unauthorized individual, report the event to the upstream cloud.

Environmental conditions also threaten the security of edge devices. “Ingress Protection” or IP rating is an industry-standard used to measure how a device will function under various

environmental conditions. The rating is a two-digit number, with the first digit indicating how well the device will protect against foreign particles such as dust. The second digit describes how the device will protect against moisture, such as indirect rain, exposure to high-pressure water, or total immersion. For example, an IP67 rating provides full dust protection through a vacuum seal and can survive up to 30 minutes when submerged in water up to a depth of 1 meter.

To ensure maximum security, all data on the device should be encrypted and protected with the strongest possible security measures. This is particularly crucial for sensitive information such as user credentials, SIM identification, and financial data. These types of credentials should be stored in a separate and secure environment to prevent unauthorized access. All ports and sensors that are not being actively used by the device should also be disabled or removed entirely if possible. (Kumari et al., 2022, ch. 4)

5.3.2 Software Security

Attackers are always looking for the “weakest link” in a given system. Therefore, architects of the Edge have to pay particular attention to software security. Edge systems are vulnerable to less secure pathways that have already been fixed in modern IT systems due to the wide range of devices and operating systems used. This provides an opportunity for less sophisticated exploits to target these weaker and less secure edge systems. It is crucial to ensure that modern edge computing devices are equipped with the latest security methods to protect themselves against such attacks.

It is crucial to have a patching and update mechanism in place for all edge systems. The system design should anticipate the need for software updates in the field. Software updates are essential to add new features, fix bugs, and provide security patches. It is necessary to ensure that all software components on a machine, including boot code, operating systems, and runtime applications, can be upgraded.

A crucial element in developing secure software is to implement secure event logging within the system. Telemetry data should be securely transmitted to a central cloud agent to identify anomalies and changes in usage patterns. Typically, edge systems exhibit a predictable pattern of CPU usage, memory usage, and network traffic. Statistical and machine learning methods can be used to detect any deviations from the norm.

And like with the removal of unused ports in hardware, software, too, should only deploy what is absolutely necessary for field operation. Adding unneeded software libraries, tools, and interfaces widens the attack vector, giving attackers more opportunities to exploit a system. (Lea, 2023, ch. 8.3)

5.3.3 Network Security

Ensuring secure networking and communication services is crucial. The best practice is to encrypt and protect all data from its origin to its destination. In modern systems, plain text data should be avoided as most hardware now has fast AES encryption mechanisms that can secure all data. Data in transit is vulnerable to interception and tampering. Therefore, it is necessary to assume that all data leaving an edge device is insecure unless explicitly secured by the engineer.

This principle of security is called Zero-Trust. Zero Trust ensures that user identity, device credentials and authenticity, geographic location, firmware versions, operating system, and even certain applications installed or removed are all validated before allowing access to the network. The Zero Trust approach ensures that the user meets all the security criteria before they can access any network resource. (Kumari et al., 2022, ch. 4)

6. Case study: Herohero edge prototype

6.1 Company background

Herohero is an online platform where anyone can create their own profile and build a community around it. The subscription-based model allows creators to freely share and monetize any type of content: videos, podcasts, photography, blog posts, etc. Furthermore, Herohero users can also use the platform to communicate directly with other creators and fans alike.

Herohero was founded in 2019 and is now being used by more than 4,000 creators and has over 75,000 paying customers. The author of this thesis is the Frontend Lead at Herohero.

6.2 Current technology stack

Herohero is a Single-Page Application (SPA), which means loading only one web page and dynamically updating the content within the same document using JavaScript APIs like XMLHttpRequest and Fetch as needed. The backend lives in the Google Cloud ecosystem and uses Fastly to cache the pages for faster load times. The backend is primarily written in Kotlin, while the frontend is written in TypeScript, a superset of JavaScript.

Frontend tool Vue, a progressive and versatile framework for building web user interfaces, is also used. The results of API calls are stored in Pinia, a persistent state management package, which caches them on the client side. During the development and build process, Vite is used, together with Vitest and Playwright, for testing the application. Vue-router is used for routing.

Both the backend and frontend are kept in a private GitLab repository. The application has localized versions in Czech and English, with plans to expand to other languages in the near future.

6.3 Analysis of possible approaches

The author sat down with Vojtěch Knyttl, the CTO, and Vojtěch Otevřel, the CEO, in early March to discuss the possibilities of introducing Web-based Edge technologies to the company's code base. The current technologies and architecture were examined, and a suitable area in which the project would benefit from Edge computing was selected.

The idea was to create a prototype application by utilizing Cloudflare Workers to pre-render the page on the user's initial visit. This would mimic the behavior of some popular full-stack frameworks without the need to rewrite the existing code to be compatible with any particular framework. This solution is lightweight while delivering the key benefits of Edge computing in web applications, namely:

- Faster page loads.
- Generating meta tags available for search engine scrapers to consume.
- Caching the responses to mitigate server load.

6.4 Implementation

With Cloudflare Workers, one can deploy serverless code instantly across the globe. (Cloudflare, 2023a) The entire Worker's code, which is 90 lines long, can be found in **Appendix A**. Here are the key parts of that code annotated.

```
return `
<title>${title}</title>
<meta property="og:title" name="title" content="${title}">
<meta property="og:description" name="description" content="${bio}" />
<meta property="og:image" content='${ogImageUrl}' />
<meta property="og:type" content="website">
<meta property="twitter:card" content="summary_large_image">
<meta property="twitter:image" content="${ogImageUrl}">
<meta itemprop="image" content="${ogImageUrl}">
<script id="creator-ld" type="application/ld+json">
${JSON.stringify({
  "@context": "https://schema.org",
  "@type": "Person",
  "name": escapeHtml(user.name),
  "image": ogImageUrl,
  "description": bio,
  "url": `https://${HOST}/${user.path}`
})}
</script>
`;
```

This code block generates the required meta tags for Search Engine Optimization (SEO). Properties with the prefix *og:* are used by Facebook, while Twitter also has its own prefixed properties for showing rich links. This is done to increase visibility and organic traffic. A higher ranking on the search engine results page (SERP) leads to increased visibility and, in turn, attracts more visitors from search engines.

```
const isNotExistingRoute = ![ '/create', '/search', '/' ]
```

```
.find(pathname => pathname === url.pathname);  
const isUserRoute = url.pathname.match(/^\/[a-z0-9]+$/);
```

`IsNotExistingRoute` matches all routes which serve a different purpose than showing a user profile, and `isUserRoute` uses a regular expression to determine that the path is correct. Like this, *herohero.co/create* gets you to the “create a new post” page while allowing other URLs on the same level to be used as individual user profiles, like *herohero.co/cestmir*, without the need for an extra segment in the URL, like *herohero.co/user/cestmir*.

```
return new Response(htmlWithMeta, {  
  headers: {  
    ...htmlHeaders,  
    "X-Rendered-By": "Cloudflare Edge SSR",  
    "Content-Type": "text/html",  
    "Surrogate-Key": `Service/SSR User/${user.id}`,  
    "Surrogate-Control": `must-revalidate, max-age=${6 * 60}`, // 6 hours  
  },  
});
```

This is where the actual HTTP Response is returned. Notice that the response also gets cached for the next 6 hours after the initial request with the use of *must-revalidate*, meaning when a user visits the page after the limit has expired, the cached version will still be served, but a new Request is generated in the background. On the next page visit, a fresh version is shown (to the requesting user, but also to all other users).

6.5 Cloudflare Workers results

With the prototype completed and tested, it was decided that since the benefits for the business were analyzed and deemed important, and the new infrastructure would not be too difficult to maintain, the application would be released into production. The implementation of Cloudflare Workers was deployed to production at the beginning of April.

Metrics were aggregated between the 1st of May and the 7th of May (one week), across all routes invoking the Worker, in this case, the configured *ssr-prod.heroheroco.workers.dev* route. They show that a total of 238,000 unique requests were made to the Worker, with all of them returning a successful response. The median response time was 1.2 milliseconds, the 75th percentile was 1.3ms, the 99th percentile was 3.2ms, and the 99.9th percentile was 4.5ms. The Worker could run free of charge as it had not surpassed the limit of 10 ms of the Free Model. The total duration of execution amounted to 1,740 GB-sec.

The average page load took 514ms pre-implementation. That figure has seen a decrease of 35.6 %, down to an average of 331ms. The page load was measured using Chrome Lighthouse,



Figure 6.1: Metrics aggregated across all routes invoking the Herohero SSR Worker. (Author’s work)

a profiling tool built into the Google Chrome browser. A total of 20 measurements were conducted for both states, with and without Cloudflare Workers.

Table 6.1: Results of Google Chrome Lighthouse profiling (Author’s work)

Measurement	Pre-implementation	With Cloudflare Workers
#1	481ms	320ms
#2	535ms	344ms
#3	492ms	326ms
#4	510ms	338ms
#5	530ms	318ms
#6	519ms	352ms
#7	497ms	325ms
#8	544ms	348ms
#9	511ms	329ms
#10	500ms	322ms
#11	533ms	334ms
#12	505ms	330ms
#13	525ms	341ms
#14	503ms	319ms
#15	515ms	337ms
#16	538ms	347ms
#17	502ms	315ms
#18	518ms	332ms
#19	528ms	342ms
#20	512ms	323ms
Average	514ms	331ms

To verify that the Workers were working as intended across regions, a VPN service has been used to simulate requests originating from across the world. The following table compares the VPN location with the header *x-served-by*, which indicates which Edge server has processed that request. Based on the header, a Cloudflare Data Center Location is matched with the value of the header. The header can be retrieved through a simple command.

```
curl -I https://herohero.co
```

Table 6.2: Comparison of VPN location with the Edge server location. (Author’s work)

VPN location	x-served-by header	CF Data Center Location
Prague, CZ (no VPN)	cache-vie6349-VIE	Vienna, AT
United Kingdom	cache-lcy-eglc8600024-LCY	London, GB
South Africa	cache-jnb7023-JNB	Johannesburg, ZA
Argentina	cache-eze2230077-ARG	Buenos Aires, AR
United States	cache-yyz4575-ATL	Atlanta, US
Australia	cache-syd10125-SYD	Sydney, AU
Japan	cache-nrt-rjtf7700042-NRT	Tokyo, JP

6.5.1 Potential drawbacks

The initial version of Amazon Prime Video monitoring service was a distributed system orchestrated by AWS Step Functions. By transitioning from a distributed microservices architecture to a monolithic application, Amazon achieved greater scalability, resilience, and cost reduction. The primary limitation to scaling within the architecture stemmed from the orchestration management: the Edge architecture was just simply too complicated and required unnecessary computation steps and network calls. (Kolny, 2023)

In the case of the Herohero Edge Workers, the architecture is very simple, requiring just 90 lines of code. Therefore, the Edge makes sense. However, the company should be weary of potentially expanding the functionality of the Workers and making the system too complex. The costs and effectiveness could negate the benefits gained by the Edge infrastructure.

Conclusion

The topic of Edge Computing was comprehensively covered, achieving the goal of investigating the application of Edge Computing to different application domains. Domains covered include smart cities, with projects such as Autonomous Security Robots and Trainspotting, healthcare, and agriculture with the AI-Powered Tomato Pollinator, and linked closely to the focus of this thesis, web applications. Netflix Open Connect was a prime example of a business case requiring an Edge solution. Serverless functions from Vercel, Deno, and Cloudflare are important parts of today's internet infrastructure, with Cloudflare Workers being utilized as a solution to the practical part of the thesis.

To answer the questions laid out in the Introduction:

- The purpose of using Edge Computing is to move data processing closer to the users, making the whole process faster and more secure. The typical business case characteristics for which an Edge solution proves useful are that it is usually a large distributed problem involving tens or even thousands of devices all needing to process data in near real-time. It is important to mention that while Edge Computing can greatly help with processing, it is not meant to replace the Cloud but rather to enhance it.
- In Web applications, the main benefits of utilizing the Edge for deployment include reduced latency for users across the globe, the ability to tailor the website's experience without needing a central server to process such requirements, and the added layer of security and resistance a multi-region deployment offers. Some drawbacks include the slightly higher time of development of such systems, as well as the risk of making the infrastructure too complicated and thus negating the positive effects of the Edge.

Edge Computing is still a novel topic, and new applications and use cases will surely be explored. If the cost of connectivity and Edge computers gets lower, which the trend suggests it will, Edge computing could be utilized in detecting the deficiencies in concrete structures, for example, and gives an alert in case of an impending collapse. Developments in the field of batteries and energy harvesting could also power small IoT devices, with solar looking like the likeliest power source.

And finally, video gaming and entertainment services are areas to watch for. Netflix has led the way by employing edge computing and CDNs to ensure low latency and high-quality video streaming. The rise of online video gaming has transformed the gaming industry. But gaming still poses unique challenges for developers. Unlike movies, which are pre-compressed and can be streamed seamlessly, video games require rendering and generating new frames 60+ times per second. Additionally, games must respond to user inputs from controllers with minimal delay to ensure smooth gameplay. These bidirectional interactions and strict latency requirements call for further improvements in edge systems and CDN networks. I can see this area being explored and new businesses could capitalize on this emerging technology.

Future works could expand on these future developments, whether they become a reality or not. Additionally, deeper research into other domains different from Web applications could be explored, as I found the solutions in agriculture and healthcare particularly interesting.

Bibliography

- AMAZON WEB SERVICES, INC., 2023. *What is Edge Computing? - Edge Computing Explained - AWS* [Amazon Web Services, Inc.] [online]. [visited on 2023-04-24]. Available from: <https://aws.amazon.com/what-is/edge-computing/>.
- ARMSTRONG, Martin, 2023. *Infographic: Netflix is Responsible for 15% of Global Internet Traffic* [Statista Infographics] [online]. [visited on 2023-05-06]. Available from: <https://www.statista.com/chart/15692/distribution-of-global-downstream-traffic>.
- BARAN, P., 1964. On Distributed Communications Networks. *IEEE Transactions on Communications* [online]. Vol. 12, no. 1, pp. 1–9 [visited on 2023-04-25]. ISSN 0096-2244. Available from DOI: 10.1109/TCOM.1964.1088883.
- BASHE, C. J., BUCHHOLZ, W., HAWKINS, G. V., INGRAM, J. J. and ROCHESTER, N., 1981. *The Architecture of IBM's Early Computers* [online]. [visited on 2023-04-28]. Available from: <https://pdfs.semanticscholar.org/58e7/9f00e2e012ac64de130982fc3b991284a50a.pdf>.
- BLOOMBERG MEDIA STUDIOS and IBM, 2020. How Smarter Businesses Transform in an Uncertain World. Available also from: <https://www.ibm.com/downloads/cas/D3J52GQP>.
- BLUES WIRELESS, 2021. *Machine Learning Models on Raspberry Pi* [Blues Wireless] [online]. [visited on 2023-04-30]. Available from: <https://blues.io/videos/machine-learning-at-the-edge-on-the-raspberry-pi-on-demand-webinar/>.
- BUMGARDNER, Cody and HICKEY, Caylin, 2020. *Making Sense of Edge Computing*. Shelter Island, NY: Manning Publications Co. ISBN 978-1-61729-862-2.
- CLOUDFLARE, 2019. *What is the Mirai Botnet?* [Cloudflare] [online]. [visited on 2023-05-02]. Available from: <https://www.cloudflare.com/learning/ddos/glossary/mirai-botnet/>.
- CLOUDFLARE, 2022. *Molin · Built with Workers · Cloudflare Workers®* [online]. [visited on 2023-05-01]. Available from: <https://workers.cloudflare.com/built-with/projects/molin>.
- CLOUDFLARE, 2023a. *Cloudflare Global Network / Data Center Locations* [Cloudflare] [online]. [visited on 2023-05-08]. Available from: <https://www.cloudflare.com/network/>.
- CLOUDFLARE, 2023b. *Cloudflare Workers®* [online]. [visited on 2023-05-08]. Available from: <https://workers.cloudflare.com/>.
- DAHL, Ryan, 2022. *JavaScript Containers* [online]. [visited on 2023-05-05]. Available from: <https://tinyclouds.org>.
- DENO, 2023. *Deno Deploy / Deno* [online]. [visited on 2023-05-08]. Available from: <https://deno.com/deploy>.
- FISHER-OGDEN, Philip, 2021. *Netflix's Cloud Edge Architecture* [LinkedIn] [online]. [visited on 2023-04-27]. Available from: <https://www.linkedin.com/pulse/netflixs-cloud-edge-architecture-philip-fisher-ogden/>.

- FLORES, Mona, 2022. *Deploying Real-Time AI Risk Prediction for Kidney Patients* [NVIDIA Blog] [online]. [visited on 2023-04-30]. Available from: <https://blogs.nvidia.com/blog/2022/06/28/kidney-dialysis-heart-failure-risk-ai/>.
- GILL, Bob and SMITH, David, 2018. The Edge Completes the Cloud: A Gartner Trend Insight Report.
- GRAHAM-CUMMING, John, 2017. *Incident report on memory leak caused by Cloudflare parser bug* [The Cloudflare Blog] [online]. [visited on 2023-05-02]. Available from: <http://blog.cloudflare.com/incident-report-on-memory-leak-caused-by-cloudflare-parser-bug/>.
- HAMILTON, Eric, 2018. *What is Edge Computing: The Network Edge Explained* [Cloudwards] [online]. [visited on 2023-04-24]. Available from: <https://www.cloudwards.net/what-is-edge-computing/>.
- HEWLETT PACKARD ENTERPRISE, 2021. *One giant leap for edge computing – OrbitsEdge case study* [PSNow] [online]. [visited on 2023-04-30]. Available from: https://www.hpe.com/psnow/doc/a50003441enw.pdf?jumpid=in_pdfviewer-psnow.
- CHAMBERS, Brian, 2023. *Enterprise Restaurant Compute* [Medium] [online]. [visited on 2023-04-30]. Available from: <https://medium.com/chick-fil-atech/enterprise-restaurant-compute-f5e2fd63d20f>.
- IBM, 2023. *What is digital transformation? / IBM* [online]. [visited on 2023-04-27]. Available from: <https://www.ibm.com/topics/digital-transformation>.
- JIANG, Andy, 2022. *The Future of the Web is on the Edge* [Deno Blog] [online]. [visited on 2023-05-08]. Available from: <https://deno.com/blog/the-future-of-web-is-on-the-edge>.
- KANE, Gerald C., 2017. *‘Digital Transformation’ Is a Misnomer* [MIT Sloan Management Review] [online]. [visited on 2023-04-28]. Available from: <https://sloanreview.mit.edu/article/digital-transformation-is-a-misnomer/>.
- KOLNY, Marcin, 2023. *Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%* [Prime Video Tech] [online]. [visited on 2023-05-08]. Available from: <https://www.primevideotech.com/video-streaming/scaling-up-the-prime-video-audio-video-monitoring-service-and-reducing-costs-by-90>. Section: Video Streaming.
- KONTZER, Tony, 2021. *Arugga’s AI-Powered Tomato Pollinator Gives Bees a Break* [NVIDIA Blog] [online]. [visited on 2023-04-30]. Available from: <https://blogs.nvidia.com/blog/2021/08/23/arugga-tomato-pollinator-bumblebees/>.
- KUMARI, K. Anitha, SADASIVAM, G. Sudha, DHARANI, D. and NIRANJANAMURTHY, M., 2022. *Edge computing: fundamentals, advances and applications*. First edition. Boca Raton, FL: CRC Press. Advances in industry 4.0 and machine learning. ISBN 978-1-00-048362-8 978-1-00-323094-6.
- LEA, Perry, 2023. *Edge Computing Technology and Applications*. S.l.: O’Reilly Media. ISBN 978-1-63343-849-1. OCLC: 1371747993.

- LEE, Angie, 2022a. *EMVisions lightweight brain-scanning device diagnose brain stroke in minutes* [NVIDIA] [online]. [visited on 2023-05-08]. Available from: <https://resources.nvidia.com/en-us-jetson-success/emvision-portable-br>.
- LEE, Angie, 2022b. *Train Spotting: Startup Gets on Track With AI and NVIDIA Jetson to Ensure Safety, Cost Savings for Railways* [NVIDIA] [online]. [visited on 2023-05-01]. Available from: https://resources.nvidia.com/en-us-jetson-success/rail-vision-startup-uses?lx=XRDs_y.
- LIVINGINTERNET, 2020. *ARPANET, Internet* [LivingInternet] [online]. [visited on 2023-04-24]. Available from: https://www.livinginternet.com/i/ii_arpanet.htm.
- MAHLER, Charles, 2023. *7 Hardware Devices for Edge Computing Projects in 2023* [The New Stack] [online]. [visited on 2023-04-30]. Available from: <https://thenewstack.io/7-hardware-devices-for-edge-computing-projects-in-2023/>.
- MICROSOFT, 2023. *IoT Edge / Cloud Intelligence / Microsoft Azure* [online]. [visited on 2023-05-08]. Available from: <https://azure.microsoft.com/en-us/products/iot-edge>.
- MOLLIK, E., 2006. Establishing Moore's Law. *IEEE Annals of the History of Computing*. Vol. 28, no. 3, pp. 62–75. Available from DOI: 10.1109/MAHC.2006.45.
- NVIDIA, 2023. *Korea's DOGU makes Iroi and Patrover, indoor and outdoor autonomous security robots*. [NVIDIA] [online]. [visited on 2023-05-01]. Available from: <https://resources.nvidia.com/en-us-jetson-success/jetson-1>.
- PRESS, Rambus, 2016. *Understanding Dennard scaling* [Rambus] [online]. [visited on 2023-05-08]. Available from: <https://www.rambus.com/blogs/understanding-dennard-scaling-2/>.
- PRINCE, Matthew, 2017. *Quantifying the Impact of "Cloudbleed"* [The Cloudflare Blog] [online]. [visited on 2023-05-02]. Available from: <http://blog.cloudflare.com/quantifying-the-impact-of-cloudbleed/>.
- RAUCH, Guillermo, 2022. *AWS and Vercel: Accelerating innovation with serverless computing – Vercel* [online]. [visited on 2023-05-01]. Available from: <https://vercel.com/blog/aws-and-vercel-accelerating-innovation-with-serverless-computing#take-the-washington-post..>
- RUPP, Karl, 2018. *42 Years of Microprocessor Trend Data / Karl Rupp* [online]. [visited on 2023-04-28]. Available from: <https://www.karlrupp.net/2018/02/42-years-of-microprocessor-trend-data/>.
- SAVER, Jeffrey L., 2006. Time Is Brain—Quantified. *Stroke* [online]. Vol. 37, no. 1, pp. 263–266 [visited on 2023-05-02]. ISSN 0039-2499, ISSN 1524-4628. Available from DOI: 10.1161/01.STR.0000196957.55928.ab.

- SINHA, Debraj, 2022. *Metropolis Spotlight: Bluecity Combines Vision AI and Lidar for Real-Time Road Safety and Traffic Congestion* [NVIDIA Technical Blog] [online]. [visited on 2023-04-30]. Available from: <https://developer.nvidia.com/blog/metropolis-spotlight-bluecity-combines-vision-ai-and-lidar-for-real-time-road-safety-and-traffic-congestion/>.
- TAYLOR, Petroc, 2021. *Total data volume worldwide 2010-2025* [Statista] [online]. [visited on 2023-04-26]. Available from: <https://www.statista.com/statistics/871513/worldwide-data-created/>.
- THE LINUX FOUNDATION, 2023. *Our Platform / EdgeX Foundry, The Open Source Edge Platform* [<https://www.edgexfoundry.org/software/platform/>] [online]. [visited on 2023-05-02]. Available from: <https://www.edgexfoundry.org/software/platform/>.
- TUNG, Teresa, 2021. *What You Need to Know About Edge Computing / Accenture* [Cloud Computing] [online]. [visited on 2023-04-24]. Available from: <https://www.accenture.com/us-en/blogs/cloud-computing/what-you-need-to-know-about-edge-computing>.
- USRDS, 2023. *Annual Data Report* [USRDS] [online]. [visited on 2023-05-08]. Available from: <https://adr.usrds.org/>.
- VAILSHERY, Lionel Sujay, 2022. *IoT connected devices worldwide 2019-2030* [Statista] [online]. [visited on 2023-04-26]. Available from: <https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/>.
- VERCEL, 2023. *Edge Functions - Vercel* [online]. [visited on 2023-05-04]. Available from: <https://vercel.com/features/edge-functions>.

Appendices

A. Cloudflare workers SSR code

```
import escapeHtml from 'escape-html';

addEventListener('fetch', (event) => {
  event.respondWith(handleRequest(event.request));
});

function getUserMetaHTML({ user }) {
  const shortBio = escapeHtml(String(user.bio).slice(0, 250));
  const ogImageUrl =
    'https://storage.googleapis.com/herohero-ogimages/${user.id}.png?${Date.now()}';
  const title = escapeHtml(`${user.name} | Herohero`);

  return `
<title>${title}</title>
<meta property="og:title" name="title" content="${title}">
<meta property="og:description" name="description" content="${shortBio}" />
<meta property="og:image" content='${ogImageUrl}' />
<meta property="og:type" content="website">
<meta property="twitter:card" content="summary_large_image">
<meta property="twitter:image" content="${ogImageUrl}">
<meta itemprop="image" content="${ogImageUrl}">
<script id="creator-ld" type="application/ld+json">
  ${JSON.stringify({
    "@context": "https://schema.org",
    "@type": "Person",
    "name": escapeHtml(user.name),
    "image": ogImageUrl,
    "description": shortBio,
    "url": `https://${HOST}/${user.path}`
  })}
</script>
`;
}

async function handleRequest(request) {
  const url = new URL(request.url);
  const isNotExistingRoute = ![ '/create', '/search', '/' ]
    .find(pathname => pathname === url.pathname);
  const isUserRoute = url.pathname.match(/^\/[a-z0-9]+$/);
  url.host = HOST_FRONTEND_SERVICE;
```

```
const fetchHtml = () => fetch(url);
if (isNotExistingRoute && isUserRoute) {
  const userPath = url.pathname.replace('/', '');
  try {
    let htmlHeaders;
    const [htmlResponse, userJSONResponse] = await Promise.allSettled([
      fetchHtml().then(response => {
        htmlHeaders = Object.fromEntries(response.headers.entries());
        return response.text();
      }),
      fetch(`https://${HOST_USER_SERVICE}/v1/users?path=${userPath}`)
        .then(response => response.json())
    ]);

    const user = userJSONResponse.value.users[0];
    if (htmlResponse.status === 'fulfilled' &&
      (userJSONResponse.status === 'rejected' || !user)) {
      return new Response(htmlResponse.value, {
        headers: {
          ...htmlHeaders,
          "X-Rendered-By": "Fastly Edge SSR",
          "Content-Type": "text/html"
        },
      });
    }
    const meta = getUserMetaHTML({ user });
    const htmlWithMeta = htmlResponse.value
      .replace(/<!-- <Meta> -->[\S\s]*<!-- <\Meta> -->/, meta);
    return new Response(htmlWithMeta, {
      headers: {
        ...htmlHeaders,
        "X-Rendered-By": "Cloudflare Edge SSR",
        "Content-Type": "text/html",
        "Surrogate-Key": `Service/SSR User/${user.id}`,
        "Surrogate-Control": `must-revalidate, max-age=${6 * 60}`, // 6 hours
      },
    });
  } catch (e) {
    console.error(e);
    const genericHtmlResponse = await fetchHtml();
    return genericHtmlResponse;
  }
} else {
```

```
// We append the X-Rendered-By header for debugging purposes.  
const genericHtmlResponse = await fetchHtml();  
return genericHtmlResponse;  
}  
}
```